AVL e Rubro-Negra

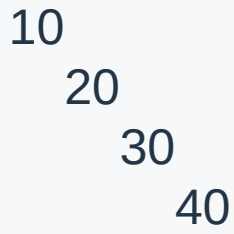
Implementação, testes e recomendação técnica

Guilherme Soares
Estruturas de Dados Básicas II

O problema: buscar sem depender da ordem de chegada

Uma árvore de busca comum mantém valores menores à esquerda e maiores à direita. Se os IDs chegam em ordem crescente, ela pode se tornar uma cadeia.

Exemplo: inserções 10, 20, 30, 40



4 nós → altura 4

Buscar 40 percorre os 4 nós.

O balanceamento limita a altura e o caminho de busca.

O protótipo e as decisões de implementação

Programa em C++17 com escolha da estrutura ao iniciar. Os identificadores são inteiros únicos.

Operações

Inserção e remoção
Busca de um ID
Percurso em ordem crescente
Visualização da árvore

Decisões

Duplicado: não insere
ID ausente: informa o usuário
Árvore vazia: mostra mensagem
Dados ficam apenas em memória

AVL: fator de balanceamento

A altura usada no programa conta nós: vazia = 0; somente raiz = 1.

FB(nó) = altura(esquerda) − altura(direita)

Em cada nó, o fator deve ser −1, 0 ou +1. Após inserir ou remover, o programa atualiza as alturas no caminho de volta.

Se o fator sair desse intervalo, a árvore aplica rotações:

Esquerda-esquerda ou direita-direita: uma rotação.
Esquerda-direita ou direita-esquerda: duas rotações.

AVL: rotação após inserir 30, 20 e 10

Antes do ajuste

```
30  (FB = +2)
  /
20
/
10
```

Rotação à direita em 30

Depois do ajuste

```
    20
   /  \
  10   30
```

A ordem de busca continua 10, 20, 30. A altura cai de 3 para 2 nós.

Rubro-Negra: propriedades

[P] = preto; [V] = vermelho. As folhas sentinelas NIL são pretas.

1. Todo nó é vermelho ou preto; a raiz é preta.

2. Um nó vermelho não tem filho vermelho.

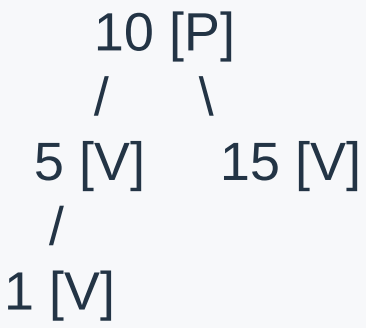
3. Todos os caminhos de um nó até as folhas NIL têm a mesma quantidade de nós pretos.

As regras limitam a altura a, no máximo, $2 \times \log_2(n + 1)$, considerando a convenção de altura em arestas.

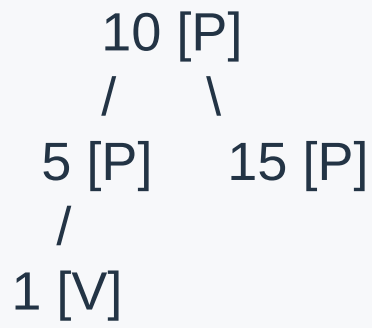
Rubro-Negra: exemplo de mudança de cores

Inserções 10, 5, 15 e 1. A inserção de 1 cria dois vermelhos consecutivos.

Antes da correção



Depois da correção

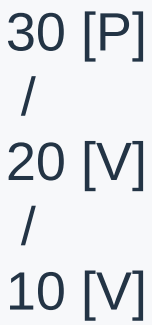


O pai e o tio eram vermelhos: ambos ficam pretos. A raiz permanece preta. Nenhuma rotação foi necessária nesse ajuste.

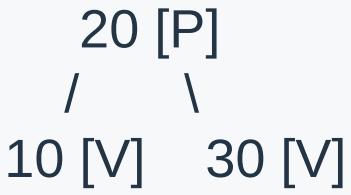
Rubro-Negra: exemplo de rotação

Inserções 30, 20 e 10. O pai é vermelho e o tio NIL é preto.

Antes



Depois



Recoloração e rotação à direita em 30 eliminam o par de nós vermelhos consecutivos. A ordem de busca é preservada.

Remoção e recuperação do balanceamento

A remoção começa como em uma árvore de busca:

Folha: o nó é retirado.
Um filho: o filho ocupa sua posição.
Dois filhos: substitui pelo sucessor em ordem e remove o sucessor.

Depois da remoção

AVL: atualiza alturas e aplica rotações quando necessário.
Rubro-Negra: se um nó preto foi retirado, corrige cores e pode realizar rotações para restaurar a altura negra.

Execução real: AVL

Trecho da saída em resultados/demonstracao_avl.txt

Inserções 30, 20 e 10: Inserido. Inserido. Inserido.

Visualização após as inserções:

```
    30 (h=1)
20 (h=2)
    10 (h=1)
```

Árvore deitada: filho direito acima da raiz.

Busca 20: Encontrado.
Remoção 30: Removido.
Percurso em ordem: 10 20

Execução real: Rubro-Negra

Trecho da saída em resultados/demonstracao_rubro_negra.txt

Inserções 30, 20 e 10: Inserido. Inserido. Inserido.

Visualização após as inserções:

```
    30 [V]
20 [P]
    10 [V]
```

[P] preto [V] vermelho

Busca 20: Encontrado.
Remoção 30: Removido.
Percurso em ordem: 10 20

Testes: operações e situações de borda

Cenário	O que foi verificado
Vazia e um nó	Busca e remoção ausentes; inserir e remover o único nó
Inserção	Crescente, decrescente, embaralhada e duplicados
Remoção	Folha, um filho, dois filhos e ID inexistente
Sequência aleatória	20.000 operações por árvore, com semente 2026

As quatro situações de rotação da AVL também foram testadas separadamente.

Testes: verificação após cada operação

O percurso em ordem foi comparado com std::set, usado somente como referência dos testes.

AVL

Ordem de busca
Altura armazenada correta
Diferença de alturas ≤ 1

Rubro-Negra

Ordem e ponteiros dos pais
Raiz e sentinela pretas
Sem vermelho com filho vermelho
Mesma altura negra por caminho

Resultado: todos os testes passaram, inclusive com verificadores de memória e comportamento indefinido.

Metodologia da comparação

Em cada cenário, as duas árvores receberam exatamente as mesmas operações:

1. Inserir 1.000 IDs: em ordem crescente, decrescente ou embaralhada (semente 42).
2. Medir altura, rotações e ajustes após as inserções.
3. Remover os primeiros 500 IDs da mesma sequência e medir novamente.

Uma rotação dupla conta como duas. Os totais no CSV são acumulados.

Resultados após 1.000 inserções

Altura em nós; rotações acumuladas. Fonte: resultados/comparacao.csv

Cenário	Altura AVL	Altura RN	Rotações AVL	Rotações RN
Crescente	10	17	990	983
Decrescente	10	17	990	983
Embaralhado	12	12	662	564

Nas entradas ordenadas, a AVL ficou 7 níveis mais baixa. Na entrada embaralhada, as alturas empataram e a RN realizou 98 rotações a menos.

Resultados após as 500 remoções

Rotações na tabela incluem inserções e remoções. Fonte: resultados/comparacao.csv

Cenário	Altura AVL	Altura RN	Rotações AVL	Rotações RN
Crescente	10	15	1238	1232
Decrescente	10	15	1238	1232
Embaralhado	11	11	874	790

Somente nas remoções: 248 rotações AVL e 249 RN nos cenários ordenados; 212 AVL e 226 RN no embaralhado.

Ajustes e interpretação dos resultados

As estruturas mantiveram alturas pequenas nos cenários testados. A AVL foi mais baixa nas entradas ordenadas; ambas empataram em altura na sequência embaralhada.

O programa também registra ajustes, mas com definições diferentes:

AVL: número de nós que precisaram de rebalanceamento.
Rubro-Negra: mudanças efetivas de cor.

Por exemplo, após inserções embaralhadas: 444 rebalanceamentos AVL e 2.313 mudanças de cor RN. Esses números não representam o mesmo tipo de operação e não devem ser comparados como custo direto.

Recomendação para o cenário investigado

Escolha proposta: AVL, se a prioridade for manter caminhos de busca curtos mesmo com IDs chegando em ordem.

Evidência

Após as inserções ordenadas, a altura foi 10 na AVL e 17 na RN; após as remoções, 10 e 15.

Alternativa

A RN teve menos rotações acumuladas nos três cenários, embora nas remoções isoladas tenha feito uma ou mais rotações.

Ambas têm operações O(log n). As rotações, isoladamente, não determinam o tempo de execução.

Limitações e referências

Os experimentos não medem tempo, memória nem tráfego real. A recomendação depende das sequências testadas; novos testes seriam necessários para produção.

Código, resultados e instruções

github.com/guilhermesoaresf/amazon-arvores-balanceadas

Fontes

Enunciado IMD0039, Unidade II.
Cormen et al., Introduction to Algorithms, 3ª ed., capítulos 12 e 13.
Open Data Structures, capítulos 9 e 10: opendatastructures.org